AIR University Islamabad

Department of Computer Electrical & Computer Engineering

PROJECT REPORT

Real-Time Object Detection System

Using Computer Vision and YOLOv8

Submitted by

Muneeba Zaher (230617)
Eiman Naeem (230603)
Zaima Ansar (233050)

Course: Applied Robotics

Submitted to: Dr, Jehanzeb Khan

1. Abstract

This project presents a Real-Time Object Detection System developed using Python, OpenCV, and the YOLOv8 (You Only Look Once version 8) deep learning framework. The system is capable of detecting and classifying multiple objects from a live webcam feed simultaneously, displaying bounding boxes, confidence scores, and object labels in real time. Key features include screenshot capture, video recording, adjustable confidence thresholds, a CSV-based detection log, and an automated warning system for potentially dangerous objects such as scissors and knives. The project leverages YOLOv8's pre-trained COCO dataset weights, demonstrating practical applications of modern computer vision in surveillance, accessibility, and smart environment monitoring.

2. Introduction

Object detection is a fundamental task in computer vision that involves identifying and localizing objects within digital images or video streams. Unlike simple image classification, which merely assigns a label to an entire image, object detection must simultaneously determine what objects are present and precisely where they are located via bounding boxes.

With the rapid advancement of deep neural networks, real-time object detection has become computationally feasible even on consumer-grade hardware. The YOLO (You Only Look Once) family of models revolutionized this field by reframing object detection as a single regression problem, enabling detection speeds that far exceed traditional two-stage detectors while maintaining competitive accuracy.

This project implements a complete, interactive detection pipeline that brings together:

- Real-time video capture via OpenCV
- YOLOv8s inference with configurable parameters
- Visual feedback through annotated bounding boxes and on-screen statistics
- Persistent logging of all detections to a CSV file
- Safety features including dangerous object warnings
- User controls for screenshots, video recording, and threshold adjustment

3. Background and Related Work

3.1 The YOLO Architecture

YOLO, first introduced by Redmon et al. in 2016, processes the entire image in a single forward pass through a convolutional neural network. The image is divided into a grid; each grid cell predicts bounding boxes and class probabilities simultaneously. This unified design eliminates the expensive region-proposal stage used by earlier detectors such as R-CNN and Faster R-CNN, dramatically reducing inference time.

YOLOv8, developed by Ultralytics in 2023, is the latest generation of this architecture. It introduces an anchor-free detection head, a refined CSPDarknet backbone, and improved training recipes. The YOLOv8s (small) variant strikes an excellent balance between speed and accuracy, making it ideal for real-time applications on laptops and standard webcams.

3.2 The COCO Dataset — YOLO's Pre-Trained Knowledge

A critical aspect of this project is understanding that the YOLOv8s model used here was not trained from scratch. Instead, it was pre-trained on the COCO (Common Objects in Context) dataset — one of the largest and most widely used benchmarks in computer vision.

The COCO dataset contains:

- Over 330,000 images collected from everyday scenes
- More than 1.5 million annotated object instances
- Precisely 80 object categories spanning people, animals, vehicles, food, household items, and more

Examples of the 80 COCO classes include: person, bicycle, car, motorbike, airplane, bus, train, truck, boat, traffic light, fire hydrant, stop sign, bench, bird, cat, dog, horse, sheep, cow, elephant, bear, zebra, giraffe, backpack, umbrella, handbag, tie, suitcase, frisbee, skis, snowboard, sports ball, kite, baseball bat, baseball glove, skateboard, surfboard, tennis racket, bottle, wine glass, cup, fork, knife, spoon, bowl, banana, apple, sandwich, orange, broccoli, carrot, hot dog, pizza, donut, cake, chair, couch, potted plant, bed, dining table, toilet, TV, laptop, mouse, remote, keyboard, cell phone, microwave, oven, toaster, sink, refrigerator, book, clock, vase, scissors, teddy bear, hair drier, and toothbrush.

Because our system uses these pre-trained weights, it can only detect objects that belong to these 80 categories. Any object outside this set — for example, a specific branded product, a medical instrument, or a culturally specific item — will either be misclassified or ignored entirely. This is a fundamental constraint that stems directly from what the model was trained to recognize.

3.3 OpenCV

OpenCV (Open Source Computer Vision Library) is an open-source library providing hundreds of computer vision and image-processing algorithms. In this project, OpenCV handles webcam capture, frame rendering, bounding box drawing, text overlay, screenshot saving, and video encoding (XVID codec via VideoWriter).

4. System Design and Architecture

4.1 High-Level Pipeline

The system follows a continuous capture-infer-display loop:

1. Frame Capture — OpenCV reads a frame from the webcam at 640×480 resolution.
2. Pre-processing — The frame is resized to 416×416 for YOLO inference (faster than the default 640×640).
3. Inference — YOLOv8s runs a single forward pass and returns bounding boxes, class IDs, and confidence scores.
4. Post-processing — Non-Maximum Suppression (IoU threshold 0.40) removes duplicate boxes. A custom correction function re-labels common misclassifications (e.g., large 'tie' detections become 'book / notebook').
5. Rendering — Annotated bounding boxes, labels, confidence scores, FPS, object count, and recording status are drawn onto the frame.
6. Logging — Each detection is appended to a CSV file with timestamp, class, confidence, and box coordinates.

7. Display — The annotated frame is shown in an OpenCV window. Optional video recording writes frames to an AVI file.

4.2 Key Configuration Parameters

Parameter	Value	Purpose
Model	yolov8s.pt	Small YOLO variant — fast inference
Confidence Threshold	0.65 (adjustable)	Minimum score for a detection to be accepted
IoU Threshold	0.40	Controls NMS duplicate suppression
Image Size	416 px	Inference resolution — smaller = faster
Camera Index	0	Default system webcam
Video Codec	XVID	Standard AVI video encoding
Log Format	CSV	Date, time, class, confidence, bounding box

4.3 Color-Coded Detection Classes

Each detected object is rendered with a distinct bounding box color to allow quick visual distinction:

- Green — person
- Magenta — book / notebook (including misclassified large objects)
- Blue — cell phone
- Red — knife or scissors (triggers the danger warning)
- Yellow — all other COCO classes

5. Implementation Details

5.1 Technologies Used

- Python 3.x — primary programming language
- Ultralytics YOLOv8 — state-of-the-art real-time object detection model
- OpenCV (cv2) — computer vision and video I/O library
- CSV module — lightweight structured detection logging
- datetime / time modules — timestamping and FPS calculation
- os module — directory management and file path handling

5.2 Module Breakdown

The codebase is organized into clearly defined functional blocks:

- Configuration block — constants for model name, thresholds, paths, and camera index
- Initialization — model loading, camera setup, directory/CSV creation
- save_detection_to_csv() — appends detection records with timestamps
- save_screenshot() — writes the current frame to a timestamped JPEG file

- start_video_recording() / stop_video_recording() — manages XVID AVI recording
- correct_common_mistakes() — heuristic relabeling based on bounding box area
- get_box_color() — returns per-class rendering colors
- draw_detection_box() — renders bounding box and label with background fill
- draw_small_text() — renders FPS, object count, confidence, and recording state
- draw_warning() — displays red danger alert for knives and scissors
- Main loop — captures frames, runs inference, coordinates all modules

5.3 Misclassification Correction

A known limitation of the COCO-trained model is that large flat rectangular objects (books, notebooks) are sometimes classified as 'tie', 'traffic light', or 'tv' due to visual ambiguity in the training data. The correct_common_mistakes() function addresses this by checking whether the bounding box area exceeds 35,000 pixels squared; if so, any of these alternate labels are overridden with 'book / notebook'. This lightweight heuristic significantly improves practical accuracy for common desktop objects.

6. Results and Demonstrations

The system was tested against a variety of household objects under standard indoor lighting conditions. The following screenshots capture representative detection results from the live webcam feed.

6.1 Cell Phone Detection

Figure 1: Cell phone detected with 92% confidence (blue bounding box). FPS: 5.

6.2 Book Detection

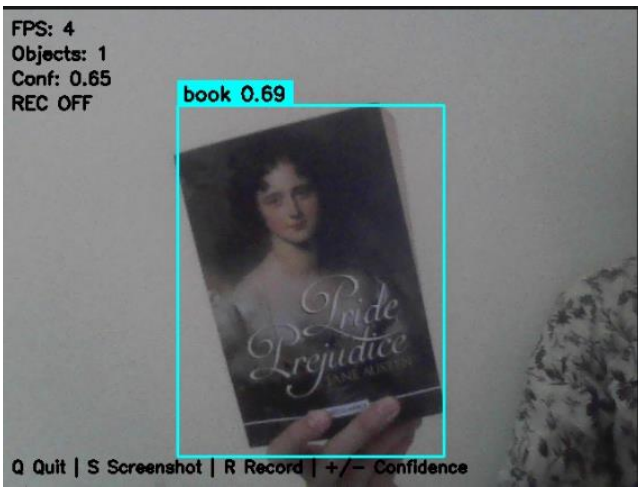

Figure 2: 'Pride and Prejudice' by Jane Austen correctly classified as 'book' with 69% confidence (cyan box).

6.3 Remote Control with Person Co-detection

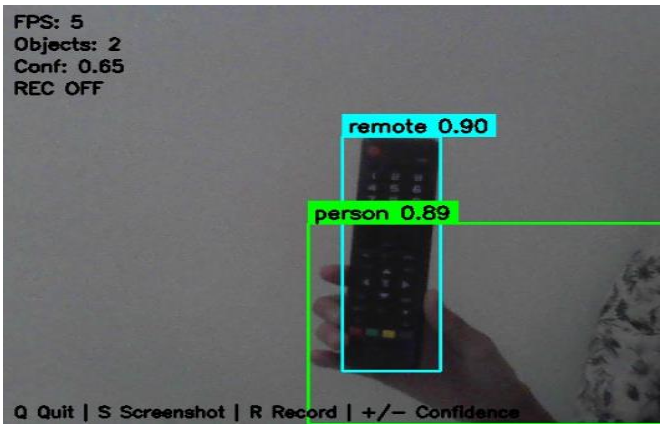

Figure 3: TV remote (90%) and person (89%) detected simultaneously. The system correctly identifies multiple objects per frame.

6.4 Notebook Detection with Heuristic Correction

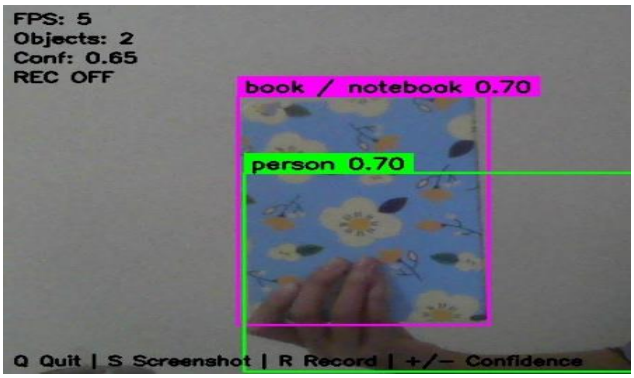

Figure 4: A floral-cover notebook re-labelled as 'book / notebook' via the area-based correction heuristic (magenta box)..

6.5 Soft Toy — Teddy Bear

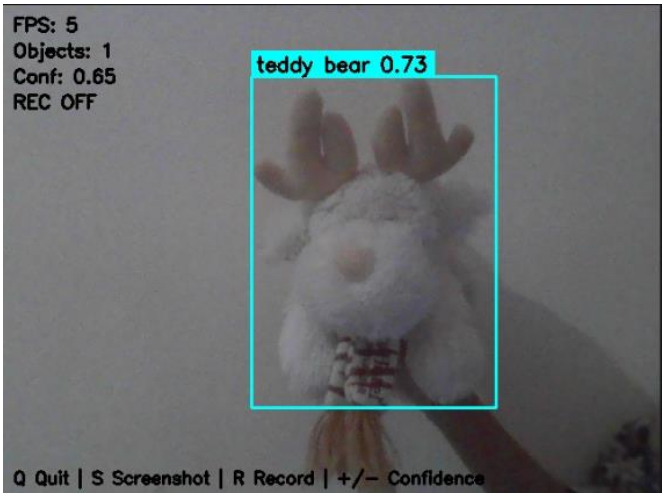

Figure 5: A reindeer plush toy classified as 'teddy bear' (73%). The COCO class 'teddy bear' covers stuffed toys broadly, so this is a reasonable match.

6.6 Scissors — Dangerous Object Warning

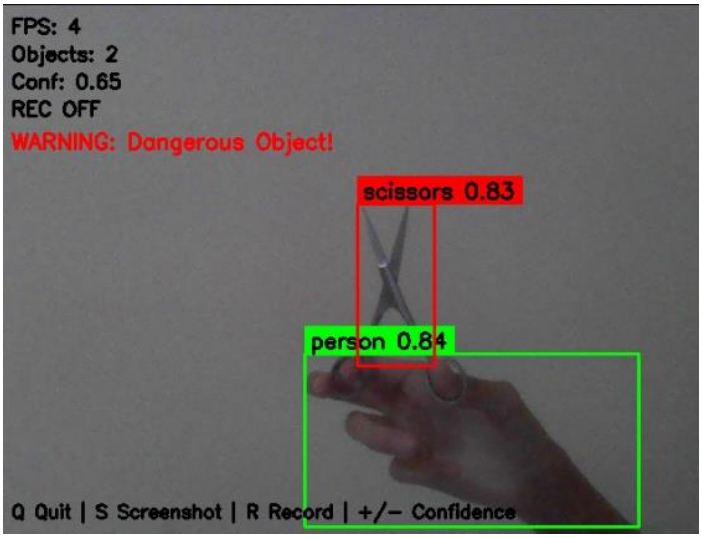

Figure 6: Scissors detected at 83% confidence (red box). The system activates the on-screen WARNING: Dangerous Object! alert and logs the event.

7. Limitations

7.1 Restricted to the COCO 80-Class Vocabulary

The most significant limitation of this system is that it can only detect objects belonging to the 80 categories in the COCO dataset on which YOLOv8s was pre-trained. We have not trained or fine-tuned the model on any custom dataset. This means:

- Objects outside the 80 COCO classes — such as specific electronic components, local cultural items, or specialized tools — cannot be detected at all.
- The model's recognition of each class reflects the distribution of that class in the COCO training images. Objects photographed from unusual angles, in poor lighting, or in unfamiliar contexts may be misdetected.
- The correct_common_mistakes() heuristic is a workaround, not a true solution. It compensates for specific known misclassifications but cannot generalize to unseen errors.

Future work should include collecting domain-specific training images, labelling them with a tool such as Roboflow or LabelImg, and fine-tuning YOLOv8 on that custom dataset to expand detection capability beyond the COCO vocabulary.

7.2 Performance Constraints

- The system achieves approximately 4–5 FPS on a standard laptop CPU. A dedicated GPU would increase throughput dramatically.
- Reducing inference resolution to 416 px improves speed at the cost of detecting small or distant objects.
- Video recording is limited to 10 FPS due to the VideoWriter configuration.

7.3 Environmental Sensitivity

- Low-light conditions (as visible in the screenshots) reduce detection confidence and can cause missed detections.
- Partial occlusion, rapid motion blur, and cluttered backgrounds all degrade accuracy.
- The single-camera setup provides no depth information, which limits potential for 3-D localization.

7.4 No Persistent Object Tracking

The system detects objects independently in each frame. There is no inter-frame tracking, so the same physical object may receive different bounding boxes or labels across consecutive frames. Integrating a tracker such as DeepSORT or ByteTrack would enable stable, ID-consistent tracking over time.

8. Future Enhancements

- Custom dataset training — collect and annotate domain-specific images, then fine-tune YOLOv8 to extend beyond COCO classes.
- Object tracking — integrate DeepSORT or ByteTrack for persistent ID assignment across frames.
- GPU acceleration — enable CUDA inference for 30+ FPS real-time performance.
- Notification system — send email or mobile alerts when dangerous objects are detected.
- Web dashboard — stream the annotated video to a browser and visualize the CSV log as live charts.
- Multi-camera support — aggregate feeds from multiple cameras for wide-area surveillance.
- Higher-resolution inference — use YOLOv8m or YOLOv8l with GPU to improve accuracy on small objects.

9. Conclusion

This project successfully implements a fully functional, interactive real-time object detection system using Python, OpenCV, and YOLOv8. The system detects and classifies everyday objects from a live webcam feed, provides visual annotations with confidence scores, logs all detections to a structured CSV file, and issues automated safety warnings for dangerous objects.

The use of YOLOv8s pre-trained on the COCO dataset enabled rapid development and strong out-of-the-box performance across 80 object categories. However, this also defines the primary limitation: detection is strictly constrained to the COCO vocabulary. We have not yet trained the model on any custom data. Extending the system to recognize domain-specific objects would require collecting labelled training images and performing transfer learning on the existing architecture.

Despite these limitations, the project demonstrates the power and accessibility of modern deep learning tools. A sophisticated, real-time vision system that would have required months of work a decade ago can now be constructed and deployed in a few hundred lines of Python. This foundation serves as a solid starting point for more advanced applications in surveillance, robotics, assistive technology, and smart environments.

10. References

- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016.
- Jocher, G. et al. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics
- Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV 2014.
- OpenCV Development Team. (2024). OpenCV — Open Source Computer Vision Library. https://opencv.org
- Ultralytics Documentation. (2024). YOLOv8 Docs. https://docs.ultralytics.com